

Structured Programming Concepts ***(26CSE0105)***



Bitwise Operator, Size of Operators in C

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

Topics to be covered

- ***Bitwise Operators***
- ***Size of Operator***
- ***Operator Precedence & Associativity***
- ***Expression Evaluation***

Bitwise Operators in C

- **Bitwise operators** in C perform operations directly on the **binary representation** of a number, performing operations **bit by bit**.
- **Bitwise operators** are widely used;
 - Work with **integral data types** like **char**, **short**, **int**, and **long**.
 - Used in **system programming**, **embedded systems**, and **bit masking**.
 - Help perform **fast operations** such as **shifting**, **toggling**, and **checking bits**.
- The **operators** are first converted to **bit-level** and then the **calculation is performed** on the **operands**.

Bitwise Operators in C

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR (XOR)
<<	Bitwise Left Shift
>>	Bitwise Right Shift
~	One's Complement

Bitwise Operators in C

Bitwise AND (&): returns **1** only when **both** corresponding bits are **1**.

For Example;

```
int a = 5;  
int b = 3;  
int c = a & b;
```

5	=	0101
3	=	0011

5&3	=	0001

Bitwise OR (|): returns **1** when **at least one** corresponding bit is **1**.

For example;

```
int a = 5;  
int b = 3;  
int c = a | b;
```

5	=	0101
3	=	0011

5 3	=	0111

Bitwise Operators in C

Bitwise XOR (^): returns **1** when the **two bits** are **different**.

For Example;

```
int a = 5;  
int b = 3;  
int c = a ^ b;
```

5	=	0101
3	=	0011

5&3	=	0110

Bitwise NOT (~): is a **unary operator** that **flips every bit** of a number:

For example;

```
int a = 5;  
~ a = -6;
```

5	=	00000101
~5	=	11110111 (use 2's complement)

- ❖ On the usual modern **2's-complement systems**.
- ❖ A useful identity is: **$\sim n = -(n + 1)$**

Bitwise Operators in C (example)

```
#include<stdio.h>
int main()
{
    int a = 12;
    int b = 5;
    printf("a & b: %d\n", a&b);
    printf("a | b: %d\n", a|b);
    printf("a ^ b: %d\n", a^b);
    printf("~a  : %d\n", ~a);
    return 0;
}
```

```
a & b: 4
a | b: 13
a ^ b: 9
~a   : -13
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Bitwise Operators in C

Left Shift (<<) Operator: moves bits **toward the left**.

For Example;

```
int a = 5;
```

```
int b = a << 2;
```

5	= 0101
5 << 2	= 010100 = 20

Right Shift (>>) Operator: moves bits **toward the right**.

For Example;

```
int a = 10;
```

```
int b = a >> 2;
```

10	= 1010
10 >> 2	= 0010 = 2

- ❖ For **+ve integer**, **<< n** is equivalent to integer multiplication of **2^n** .
- ❖ For **+ve integer**, **>> n** is equivalent to integer division by **2^n** .

Bitwise Operators in C (example)

```
#include<stdio.h>
int main()
{
    int a = 5, b = 10;
    printf("a << 2: %d\n", a << 2);
    printf("b >> 2: %d\n", b >> 2);
    return 0;
}
```

```
a << 2: 20
b >> 2: 2
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

sizeof() Operator in C

- The **sizeof()** operator is a **very useful tool** that helps programmers understand **how much memory** a **variable** or **data type** occupies, measured in **bytes**.
- It is a **compile-time unary operator**, which means the compiler calculates the size **before the program runs**.

Usage of sizeof() operator:

1. **sizeof()** operator with Data Types
2. **sizeof()** operator with Variables
3. **sizeof()** operator with Expression
4. **sizeof()** operator with Array
5. **sizeof()** operator with Pointer

❖ **%zu** is the **standard and correct format specifier** for **sizeof()**.

sizeof() Operator in C

- **sizeof() operator with Data Types:**

```
#include <stdio.h>
int main()
{
    printf("Size of char = %zu byte\n", sizeof(char));
    printf("Size of int = %zu bytes\n", sizeof(int));
    printf("Size of float = %zu bytes\n", sizeof(float));
    printf("Size of double = %zu bytes\n", sizeof(double));
    return 0;
}
```

```
Size of char = 1 byte
Size of int = 4 bytes
Size of float = 4 bytes
Size of double = 8 bytes
```

```
...Program finished with exit code 0
Press ENTER to exit console. □
```

sizeof() Operator in C

- **sizeof() operator with Variables:**

```
#include <stdio.h>int main()
{
    int a;
    float b;
    char c;
    printf("Size of variable a: %zu bytes\n", sizeof(a));
    printf("Size of variable b: %zu bytes\n", sizeof(b));
    printf("Size of variable c: %zu bytes\n", sizeof(c));
    return 0;
}
```

```
Size of variable a: 4 bytes
Size of variable b: 4 bytes
Size of variable c: 1 bytes

...Program finished with exit code 0
Press ENTER to exit console.█
```

sizeof() Operator in C

- **sizeof() operator with Expression:**

```
#include <stdio.h>int main()
{
    int a = 0;
    double d = 10.21;
    printf("Size: %zu bytes", sizeof(a + d));
    return 0;
}
```

```
Size: 8 bytes
```

```
...Program finished with exit code 0
Press ENTER to exit console.□
```

sizeof() Operator in C

- *sizeof() operator with Array:*

```
#include <stdio.h>
int main()
{
    int arr[] = {1,2,3,4,7,8,0,12,35,49};
    printf("Number of elements: %lu ",sizeof(arr)/sizeof(arr[0]));
    return 0;
}
```

```
Number of elements: 10

...Program finished with exit code 0
Press ENTER to exit console.█
```

sizeof() Operator in C

- **sizeof() operator with Pointer:**

```
#include <stdio.h>
int main()
{
    int *p;
    printf("Size of pointer: %zu", sizeof(p));
    return 0;
}
```

```
Size of p: 8
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

- Here, **sizeof(p)** gives the **size of the pointer**, not the **size of the integer** to which it points.
- On a typical 64-bit system: **sizeof(p) = 8 bytes**

sizeof() Operator in C

- **sizeof() operator with String:**

```
#include <stdio.h>
int main()
{
    char name[] = "Ramandeep";
    printf("Size of string: %zu bytes", sizeof(name));
    return 0;
}
```

```
Size of string: 10 bytes
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

- There are **9 characters/bytes**, C adds a null character '**\0**' At the end of string.

Operator Precedence in C

- While **executing an arithmetic statement** that has **multiple operators**, there might be **some concerns** about their evaluation.
- For these concerns, we need to understand;

“The **priority** or **precedence** in which the **operations** in an **arithmetic statement** are performed”.

Priority	Operators	Description
1 st	* / %	Multiplication, Division, Modular division
2 nd	+ -	Addition, Subtraction
3 rd	=	Assignment

Operator Associativity in C

- When an expression contains **two operators** of **equal priority** then **tie between them** is settled using the **associativity** of the **operators**.
- All **operators in C** have;
 - either **Left to Right associativity**.
 - or **Right to Left associativity**.

Description	Operator	Associativity
Multiplication	*	Left to right
Division	/	Left to right
Modulus	%	Left to right
Addition	+	Left to right
Subtraction	-	Left to right

Scanning & Evaluation in Compiler

- **Scanning** and **Evaluation** are **two separate stages**.

Scanning:

- **Compiler** scans the statement from **left to right** and identifies **tokens**.

Evaluation:

- **Compiler** groups and evaluates operators **according to precedence** and **associativity**; therefore, **evaluation** is not simply **left-to-right**.

- **Compiler reads/scans** the expression from **left to right** to identify tokens.
- But it **does not simply calculate** everything from **left to right**. i.e., (**operators** of **same precedence** are **grouped** from **left to right**).

Scanning Version (example)

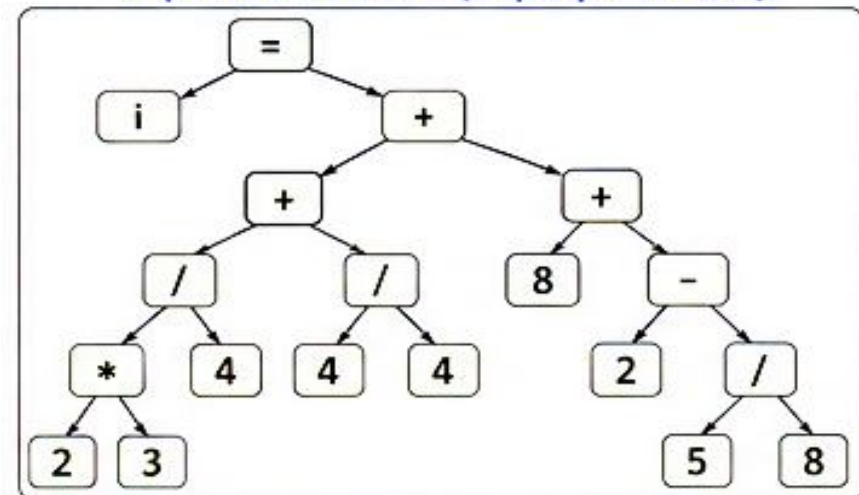
Example: *Evaluate* the following expression, assuming that *i* is an *integer variable*:

$$i = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8$$

The compiler goes through the following phases:

Phase 1 Lexical Analysis (Scanning)	Reads the source code from left to right and breaks it into tokens (lexemes).
Phase 2 Syntax Analysis (Parsing)	Checks the grammatical structure and builds the structure of the expression as per operator precedence and associativity.
Phase 3 Semantic Analysis	Checks meaning of expression, data types, type compatibility, etc.
Phase 4 Intermediate Code Generation	Converts the expression into an intermediate form (e.g., postfix).
Phase 5 Code Generation	Generates target (machine) code and executes the program.

Expression Structure (as per precedence)



Postfix (Intermediate Code)

i 2 3 * 4 / 4 4 / + 8 2 - 5 8 / + + =

Compilation Result

No errors.
Program compiled successfully. ✓

Execution Result (Value Stored)

i = 8

Evaluation Version (example)



Example: *Evaluate* the following expression, assuming that *i* is an *integer variable*:

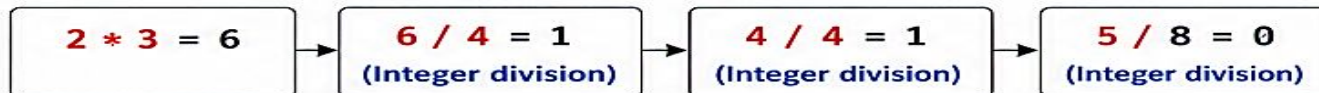
$$i = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8$$

Step 1: Apply Operator Precedence

- *** and */* have higher precedence than *+* and *-*.
- Operators with the same precedence are evaluated from left to right,

$$i = (2 * 3 / 4) + (4 / 4) + 8 - 2 + (5 / 8)$$

Step 2: Evaluate * and */* from left to right**



Step 3: Substitute the results

$$i = 1 + 1 + 8 - 2 + 0$$

Step 4: Evaluate *+* and *-* from left to right



Step	Expression	Result
1	$2 * 3$	6
2	$6 / 4$	1
3	$4 / 4$	1
4	$5 / 8$	0
5	$1 + 1$	2
6	$2 + 8$	10
7	$10 - 2$	8
8	$8 + 0$	8
Final	$i = 8$	$i = 8$

Final Result

$$i = 8$$

Important C Rule

When both operands of */* are integers, C performs **integer division**.

Thus,

$$6 / 4 \text{ gives } 1$$

$$5 / 8 \text{ gives } 0$$

Exercise (example)

Example: Swap two integers without using a third (temporary) variable by using the XOR (^) bitwise operator.

```
#include <stdio.h>
int main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d",&a,&b);
    printf("Before swapping: a = %d, b = %d\n", a, b);
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
    printf("After swapping: a = %d, b = %d\n", a, b);
    return 0;
}
```

Important XOR properties

Remember these three properties:

$$x \wedge x = 0$$

$$x \wedge 0 = x$$

$$x \wedge y \wedge y = x$$

Step 1

$$a = a \wedge b$$

Step 2

$$b = a \wedge b = (a \wedge b) \wedge b$$
$$b = a$$

Step 3

$$a = a \wedge b = (a \wedge b) \wedge a$$
$$a = b$$



Thank You!

